

# Session 3: Git and $\text{\LaTeX}$ in VS Code

## Version Control and the Integrated Research Environment

Brady Allardice

UPF & IBEI

**Session 1:** What Claude Code can do. How it works. CLAUDE.md and plan mode.

**Session 2:** The core workflow. Data cleaning, estimation, robustness,  $\text{\LaTeX}$  tables.

**Session 3:** Building infrastructure around the workflow.

Today you will:

- 1 Learn the minimum viable git workflow for agentic coding
- 2 Bring your paper into the same project directory as your code
- 3 Use Claude Code to write, compile, and version-control a  $\text{\LaTeX}$  document

Session 2 taught you to verify Claude Code's work. Today you get the tools to manage it.

Git is research insurance,  
not software engineering ceremony.

You need exactly four commands.

# What Are Git and GitHub?

**Git** is a tool that tracks every change to every file in a project. It runs on your computer. It is free and open source.

Think of it as “track changes” for your entire project directory — not just one document, but every file: code, data descriptions,  $\text{\LaTeX}$  files, everything.

**GitHub** is a website that hosts git projects online.

- It is where open-source code, replication packages, and increasingly papers live
- Git works entirely on your machine. GitHub is optional — it adds backup, sharing, and collaboration
- Today we only use git locally. GitHub comes in Session 4.



## The Key Idea

Git lets you save snapshots of your project at any point. You can always see what changed between snapshots, and go back to any

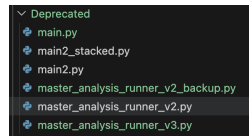
# Why Researchers Should Care About Git

## Without git, you have probably done this:

- `analysis_v2.py`, `analysis_v2_final.py`,  
`analysis_v2_final_REAL.py`
- “Which version of the cleaning script produced these results?”
- “I changed something last week and now the code is broken”

## With git:

- One file, full history. Every version is recoverable.
- You can see exactly what changed, when, and why.
- You can undo any change — even from weeks ago.



Git is useful for any research project. But when you are working with an agent that modifies your files directly, it goes from useful to **essential**.

# Why Git Matters for Agentic Coding

**The problem:** Claude Code modifies your files directly.

- It might change a file you did not ask it to change
- It might overwrite working code with something broken
- It might make 10 changes when you only wanted 3

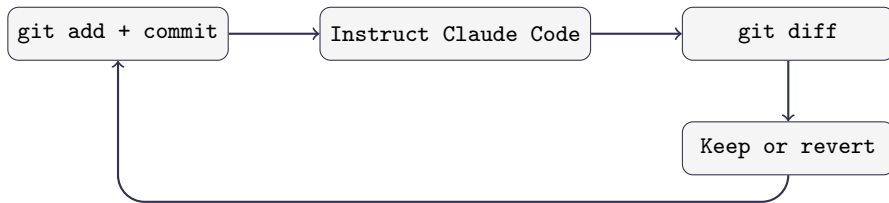
**Without git:** You try to manually undo changes. You are not sure what the file looked like before. You lose work.

**With git:** You see exactly what changed, line by line. You revert anything you do not want. You never lose work.

## The Rule

Never let Claude Code touch your project without a recent commit to fall back on.

# The Minimum Viable Git Workflow



---

## Command

## What it does

---

`git add -A && git commit -m "msg"`

Snapshot your current state

`git diff`

See what changed since last commit

`git checkout -- file.py`

Undo changes to a specific file

`git log --oneline`

See your commit history

---

## Command 1: `git init`

**What it does:** Turns a regular folder into a git project.

```
git init
```

- You run this **once**, when you start a new project
- It creates a hidden `.git/` folder that stores your project's history
- Nothing else changes — your files stay exactly as they are

Think of it as saying: “I want to start tracking this folder.”

### Note

If you cloned this course repository, git is already initialized. You can check by running `git status` — if it prints something, you are already set up.



## Command 2: `git add`

**What it does:** Selects which files to include in your next save point.

```
git add -A
```

- `-A` means “add everything that changed” — new files, modified files, deleted files
- This does not save anything yet. It just marks what will be included in the next commit.

You can also add specific files instead of everything:

```
git add scripts/analysis.py
```

Think of it as putting files in a box before sealing it. `git add` fills the box; `git commit` seals it.

## Command 3: `git commit`

**What it does:** Saves a snapshot of everything you added.

```
git commit -m "Clean merge script, verified row counts"
```

- `-m` stands for “message” — a short note describing what this snapshot contains
- The message goes in quotes. Write something your future self will understand.
- After committing, you have a permanent save point you can always return to.

**In practice, you can always run `add` and `commit` together (but you don't have to):**

```
git add -A && git commit -m "your message"
```

The `&&` just means “run the second command after the first one finishes.”

## Command 4: `git diff`

**What it does:** Shows you exactly what changed since your last commit.

```
git diff
```

The output looks like the diff view you already know from VS Code:

- Lines starting with + were added (green in VS Code)
- Lines starting with - were removed (red in VS Code)
- Everything else is unchanged context

**When to use it:** After Claude Code makes changes, run `git diff` to see *everything* it touched — not just the file it told you about.

### Tip

You can also ask Claude Code: “Run `git diff` and explain each change you made.”

## Command 5: `git checkout`

**What it does:** Undoes changes by restoring a file to its last committed state.

**Undo changes to one file:**

```
git checkout -- scripts/analysis.py
```

This throws away all changes to that file since your last commit. The file goes back to exactly how it was.

**Undo all changes (reset everything):**

```
git checkout -- .
```

The `.` means “this directory” — it reverts every file in your project.

**This is your undo button**

Claude Code changed something you did not want? One command and it is gone. This only works if you committed first — which is why you always commit before instructing.

## Command 6: `git log`

**What it does:** Shows you the history of all your commits.

```
git log --oneline
```

**Output looks like:**

```
a3f7b21 Add robustness checks  
e5c9d04 Clean merge script, verified row counts  
b1a2c33 Initial commit
```

- Each line is one commit — one save point
- The letters and numbers on the left are the commit ID
- The message on the right is what you wrote with `-m`
- Most recent commits are at the top

This is your project's timeline. You can always see what you did and when.

# Putting It Together: The Cycle

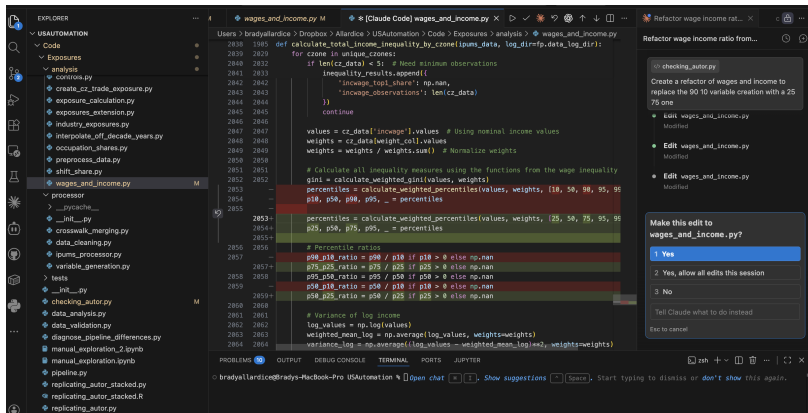
## Before every interaction with Claude Code:

- 1 `git add -A && git commit -m "message"` — save your current state
- 2 Give Claude Code an instruction
- 3 `git diff` — see what it changed
- 4 Decide:
  - Happy? → `git add -A && git commit -m "message"`
  - Not happy? → `git checkout -- .`
- 5 Repeat

## The Mindset Shift

Commits are not milestones. They are save points — like pressing Ctrl+S. Commit early, commit often, commit before every interaction with Claude Code.

# You Have Already Seen This



Red = removed, green = added. You have been reading diffs since Session 1.

**In VS Code:** When working directly in the editor, you see the git diff visually before you accept the change. This shows exactly what Claude Code changed.

# Git and Data: What Not to Track

Git is designed for **code and text files** — things that change line by line. It is *not* designed for large data files.

## Commit:

- Scripts (.py, .R, .do)
- L<sup>A</sup>T<sub>E</sub>X files (.tex, .bib)
- Documentation (CLAUDE.md)
- Small data files (under a few MB)

## Do not commit:

- Large datasets (50MB+)
- Output you can regenerate
- Sensitive data (PII, API keys)

## Rule of Thumb

Commit what a collaborator needs to *understand* your project. Skip what they could *regenerate* by running the code.



# .gitignore: Telling Git What to Skip .claudeignore: Telling Claude What to Skip

A .gitignore/.claudeignore file lists patterns for files git/claude should never track. Place it in the folder where you ran `git init`.

**Paths are relative to that folder.** Example:

```
my-project/ <- run "git init" here
.gitignore <- goes here
.claudeignore <- goes here
code/
  analysis.py
data/ <- put in .gitignore but NOT .claudeignore
  raw/
  processed/
output/ <- put in .gitignore but NOT .claudeignore
```

**Always include in .gitignore and .claudeignore:**

```
.DS_Store
.env <- usually contains sensitive information
*.pyc
__pycache__/
*.aux
*.log
```

Then add: `data/raw/`, `*.csv.gz`, any large files.

# What Ignore Files Look Like

## .gitignore

```
# Folders
data/raw/
data/*/administrative/
output/

# Specific files
.env
.DS_Store
data_merged.csv

# All Python cache
__pycache__/
```

## .claudeignore

```
# Sensitive files
.env
.DS_Store
NUCLEAR_CODES_DO_NOT_SHARE.txt
list_of_my_biggest_fears.docx

# Very large files
*.csv.gz
copy_of_internet.csv
output/

# Python cache
__pycache__/
```

**Wildcards:** \* matches any characters.

- \*.pyc = all .pyc files (anywhere in project)
- data/\*.csv = data.raw.csv, data.clean.csv, etc.
- analysis\_\*.py = analysis\_v1.py, analysis\_v2.py, etc.

**Folders** must end with / (e.g., output/, \_\_pycache\_\_/)

**Path scope:** \* = any level. \*\* = any depth. data\_merged.csv matches anywhere. data/data\_merged.csv matches only in data/.

# So Where Does the Data Go?

## Why git is bad for data:

- Git stores *every version* of every file. A 200MB CSV edited 10 times = 2GB of history.
- Git compares files line by line. Change one cell in a CSV and git sees the entire row as new.
- Large files slow down every git operation — commits, diffs, cloning.

## For most research projects (data under a few GB):

- Keep data in a synced folder: **Dropbox, OneDrive, Google Drive**
- Your code lives in git. Your data lives next to it. Your `.gitignore` keeps them separate.
- Simple, familiar, and good enough for most social science workflows

## For larger or more complex needs:

- **Object storage:** AWS S3, Google Cloud Storage — cheap, scalable
- **Data versioning:** DVC (Data Version Control) — git-like tracking for large files

You do not need these today. But if your data outgrows Dropbox, they exist.

## Exercise Part 1: Choose Your Project

### Option A: Your own research project

- Open your project folder in VS Code: File → New Window → File → Open Folder
- Use whatever code and data you have in that project

### Option B: Practice sandbox

- On your Desktop, create a new folder: `mkdir ~/Desktop/session_2_git_practice`
- Copy Session 2 materials into it: `cp -r module_2/* ~/Desktop/session_2_git_practice/`
- Open it in VS Code: File → New Window → File → Open Folder → select `session_2_git_practice` on Desktop

Pick one and open it in VS Code. Then proceed to the next slide.

## Exercise Part 1: Initialize and Commit (10 min)

Now, in your chosen project directory:

- ❶ **Create a .claudeignore FIRST:** In VS Code (File → New File → name it .claudeignore) or terminal (`echo ".env" > .claudeignore`). Add .env and any other sensitive files.  
**Important (Option B):** You will find a .env file with my real credentials. Exclude it immediately.
- ❷ **Create a .gitignore now:** Ask Claude Code to help. Include large data files, regenerated output, and system files.  
Note: .claudeignore uses the same syntax.
- ❸ **Initialize git:** `git init` then `git add -A` && `git commit -m "Initial commit"`
- ❹ **Run:** `git log --oneline`

## Exercise Part 2: The Cycle (25 min)

Do this **three times** (you decide what changes):

- ❶ **Commit** your current state
- ❷ **Instruct Claude Code** to make a change. Pick anything:
  - Add a control variable, refactor into functions, change sample restriction
  - Fix a bug, improve documentation, rename variables
  - Add a new analysis, create a visualization, write a summary
- ❸ **Run** `git diff` and read what changed
- ❹ **Decide:** keep (commit) or revert (`git checkout -- .`)

**Goal:** Try to revert at least once. See that you can always undo.

If you revert and regret it, you can recover the changes from your last commit using `git reflog` and `git checkout`.

Commit → Instruct → Diff → Decide

## Discussion:

- Did Claude Code change any files you did not expect?
- Was the diff easy to read? What was confusing?
- Did anyone revert a change? What went wrong?

## The Habit

**Commit** → **Instruct** → **Diff** → **Decide**.

This is not optional when working with an agent that modifies your files. It is the minimum responsible workflow.

In Session 4, we will automate the commit step with a hook — so you never forget.

What if your paper lived in the same place  
as your code and data?

This is the conceptual pivot of the course.



# The Problem with Overleaf

**Most researchers keep their paper in a separate tool.**

- Code lives in a project directory (or on your Desktop)
- Data lives next to the code (maybe)
- The paper lives in Overleaf (or Word, or Google Docs)

**What this means for Claude Code:**

- It cannot read your paper
- It cannot check the methods section against your code
- It cannot update a table when the analysis changes
- It cannot commit changes to your paper alongside code changes

## The Limitation

If Claude Code cannot see it, Claude Code cannot operate on it.

Your paper is the biggest piece of your project that is currently invisible.

# The Solution: $\text{\LaTeX}$ in VS Code

## Bring your paper into the project directory.

- Your `.tex` file lives alongside `scripts/` and `data/`
- Claude Code can read it, write it, and edit it — just like any other file
- Changes to the paper are tracked by git, just like code changes
- Coauthors review changes through version control, not “track changes”

## What becomes possible:

- Claude Code writes a methods section and you verify it against the code
- The analysis changes  $\rightarrow$  the table updates  $\rightarrow$  the paper updates
- Every version of the paper is recoverable through git

## The Point

This is not about  $\text{\LaTeX}$  vs. Word. It is about bringing your paper into the agentic workflow.

## What you need (should be installed from pre-work):

- ① A T<sub>E</sub>X distribution: TeX Live (Linux/Windows) or MacTeX (Mac)
- ② The **LaTeX Workshop** extension in VS Code

## What you get:

- Syntax highlighting for `.tex` files
- One-click compilation to PDF
- PDF preview in a split pane — side by side with your code
- Error messages in the VS Code terminal

## The workflow:

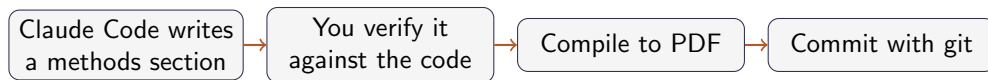
Edit `.tex` → Save → Auto-compile → PDF updates in the preview pane.

You never leave VS Code. Your paper is right next to your code.

```
\documentclass{article}
\usepackage{booktabs}
\title{My Research Paper}
\author{Your Name}
\begin{document}
\maketitle
\section{Data Description}
We use a survey of 2,044 Polish adults...
\section{Results}
\input{output/robustness_table.tex}
\end{document}
```

**Key idea:** `\input{output/robustness_table.tex}` pulls in the table from Session 2. When the table updates, the paper updates.

# Demo: The Full Loop



Everything happens in VS Code.

Everything is version-controlled.

Everything is visible to Claude Code.

## Exercise Part 3: Build a $\text{\LaTeX}$ Document (30 min)

Using the provided template:

- 1 **Commit** your current state (the habit starts now)
- 2 **Ask Claude Code** to create a `.tex` file with:
  - A data description section based on your Session 2 pipeline
  - The robustness table via `\input{output/robustness_table.tex}`
- 3 **Compile** to PDF and verify it renders correctly
- 4 **Diff and commit** the new file
- 5 **Ask Claude Code** to write a results paragraph interpreting the table. Review what it writes — would you sign off on this for a paper?
- 6 **Ask Claude Code** to check the data description against the actual cleaning code. Does the text match what the code does?

Notice: you are now using git and  $\text{\LaTeX}$  together. Every change to the paper is tracked.

# What You Built Today

- 1 **Git:** You have a safety net. Commit → instruct → diff → decide. You practiced reverting changes and saw that you can always go back.
- 2 **L<sup>A</sup>T<sub>E</sub>X in VS Code:** Your paper is now part of your project. Claude Code can see it, edit it, and check it against your code.
- 3 **The two together:** Every change to your paper is version-controlled, just like every change to your code. You used the git cycle on your `.tex` file.

## The Bigger Picture

Your project directory now contains: data, code, output, and a paper — all under version control, all visible to Claude Code. This is the foundation for everything in Session 4.

## Discussion questions:

- ❶ Did Claude Code change any files you did not expect? How did you handle it?
- ❷ Did anyone revert a change? What went wrong?
- ❸ When Claude Code wrote parts of the  $\text{\LaTeX}$  document, what did you have to correct?
- ❹ How does this compare to your current workflow — Overleaf, Word, separate directories?

## Key Takeaway

The skill is not just using Claude Code. It is building an environment where Claude Code can do its best work — and where you can verify and undo everything it does.



### Session 4: Power Features — Hooks, MCPs, and Skills

- **Hooks:** Automate the commit step — every change Claude Code makes is tracked automatically
- **MCPs:** Connect Zotero — search your library and pull citations from inside Claude Code
- **Skills:** Reusable workflows — spec validation, robustness checks, and more

#### Before next session:

- Practice the commit-instruct-diff-decide cycle on your own project
- Try asking Claude Code to review a piece of your own code
- Make sure your Zotero account is set up (instructions in the setup guide)

